

The Alberta Buck - Ethereum Implementation (DRAFT v0.1)

Perry Kundert

2026-03-07 12:00:00



This document presents a concrete Ethereum implementation of the Alberta Buck Architecture, specifying the three core smart contracts:

1. **BUCK_CREDIT** (ERC-721): An NFT representing an insurer's offer of parametric insurance on a real-world asset, with deterministic depreciation curves and piecemeal client activation.
2. **BUCK** (ERC-20): A fungible token minted against aggregated BUCK_CREDIT values, with on-chain credit-limit enforcement and default insurance premiums.
3. **BUCK_K** (Custom): An on-chain PID controller that computes a dynamic Value Stabilization Factor from commodity-basket oracle prices vs. BUCK market price, maintaining purchasing-power parity.

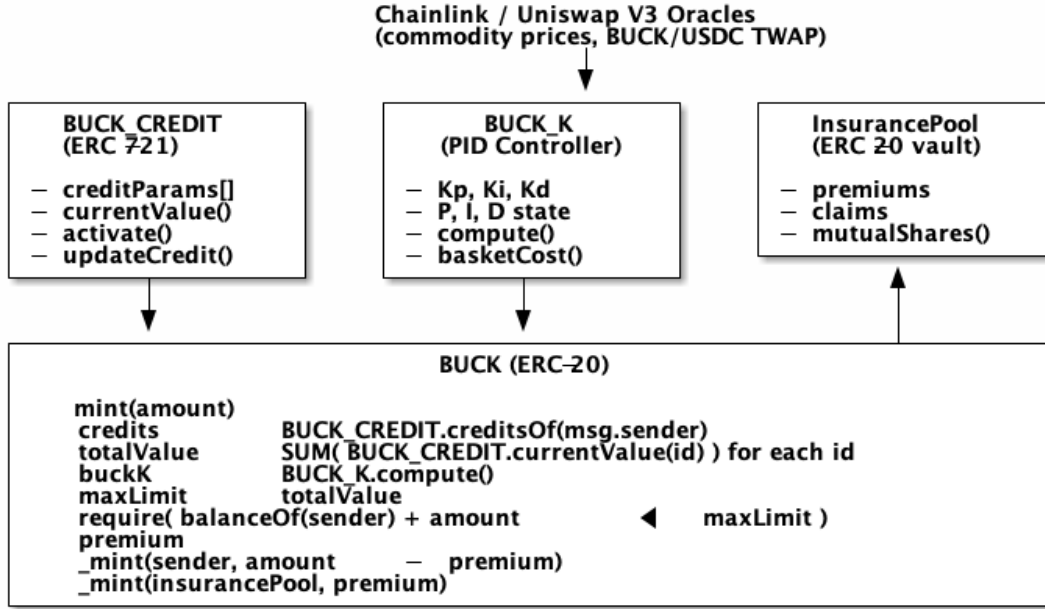
This is the first of several planned implementation approaches. The ERC-721 approach is chosen for its clarity, wide tooling support, and natural fit for unique insured assets. Subsequent documents will examine ERC-1155 (semi-fungible batch credits), ERC-3643 (regulated security token with identity compliance), and ERC-4626 (vault-based credit pooling).

Contents

1	Contract Architecture Overview	3
2	ERC Standard Analysis for BUCK_CREDIT	3
2.1	Recommendation	4
3	BUCK_CREDIT: ERC-721 Insured Asset NFT	4
3.1	Data Model	4
3.2	Depreciation Models	5
3.2.1	Depreciation Examples	7
3.3	Activation (Execution) Model	7
3.4	Insurer Updates	7
4	BUCK: ERC-20 Token	8
4.1	mint() Implementation	8
4.2	Default Insurance Premium	9
5	BUCK_K: On-Chain PID Value Stabilization Controller	10
5.1	Process Variable and Setpoint	10
5.2	Solidity Implementation	10
5.3	PID Tuning Considerations	13
5.4	Adaptive Gain Scheduling (Future Enhancement)	13
6	Integration: Full mint() Walkthrough	14
7	Gas Cost Analysis	14
8	Security Considerations	15
8.1	Oracle Manipulation	15
8.2	Credit Enumeration DoS	15
8.3	Insurer Trust	15
8.4	Reentrancy	15
9	Future Approaches	15
9.1	ERC-1155: Semi-Fungible Batch Credits	16
9.2	ERC-3643 (T-REX): Regulated Security Token	16
9.3	ERC-4626: Vault-Based Insurance Pool	16
9.4	L2 and Cross-Chain Deployment	16

1 Contract Architecture Overview

The Alberta Buck Ethereum implementation comprises three interacting contracts plus external oracle infrastructure:



The flow is:

1. An **insurer** creates a BUCK_CREDIT NFT representing their offer to insure a client's asset.
2. The **client** (asset owner) activates all or part of their credit, paying the current premium.
3. When the client calls BUCK.mint(), the contract aggregates all of the client's active credits, applies depreciation, multiplies by the current BUCK_K, and mints BUCKs up to the resulting limit.
4. BUCK_K is recomputed on-chain from commodity oracle prices vs. BUCK market price, using a PID controller that amortizes its state update across mint operations.

2 ERC Standard Analysis for BUCK_CREDIT

Before diving into the ERC-721 implementation, we compare candidate standards against the BUCK_CREDIT requirements:

Requirement	ERC-721	ERC-1155	ERC-3643 (T-REX)	ERC-4626
Unique per-asset credit	Native	Possible (NFT mode)	Native	Poor
Insurer-mutable parameters	Custom role	Custom role	Built-in agent roles	Not o
Deterministic on-chain valuation	Custom view	Custom view	Custom view	share
Piecemeal client activation	Custom state	Fungible sub-amounts	Custom state	depos
Transfer restrictions	Custom	Custom	Built-in compliance	Open
Enumerable (for mint aggregation)	ERC721Enumerable	balanceOf per ID	Inherits ERC-20	Singl
Regulatory identity / KYC	Not built-in	Not built-in	ONCHAINID native	Not b
Tooling / ecosystem maturity	Excellent	Very good	Moderate (niche)	Exce

2.1 Recommendation

ERC-721 is the strongest first choice:

- Each **BUCK_CREDIT** is genuinely unique (specific asset, depreciation curve, insured value, premium schedule). This is precisely what ERC-721 models.
- The **ERC721Enumerable** extension provides `tokenOfOwnerByIndex()`, required for `BUCK.mint()` to aggregate all credits in an account.
- Custom extensions for dual-role access (owner activates, insurer updates) and deterministic valuation are straightforward.
- Widest tooling support, most audited implementations, lowest learning curve.

ERC-3643 is the strongest future candidate, especially if regulatory compliance (KYC, transfer restrictions, identity registry) becomes a deployment requirement. Its agent-role system maps naturally to the insurer/client relationship. However, its dependency chain (ONCHAINID, compliance modules, claim topics) adds significant complexity that is better deferred.

ERC-1155 is attractive if standardized credit "classes" emerge (e.g., all Calgary residential properties with the same depreciation model), allowing semi-fungible batching. For the initial system where each credit is individually appraised, the fungibility benefit doesn't apply.

ERC-4626 is a poor fit for individual credits but may be useful for the **InsurancePool** (where mutual insurance shares *are* fungible vault interests).

3 BUCK_CREDIT: ERC-721 Insured Asset NFT

3.1 Data Model

Each **BUCK_CREDIT** token stores the insurer's current offer of insurance and the client's activation state. All monetary values are denominated in BUCK-equivalent units (18 decimals).

```
// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC721/extensions/ERC721Enumerable.sol";

contract BuckCredit is ERC721Enumerable {

    enum DepreciationType {
        NONE,           // Non-depreciating (land, gold, crypto)
        LINEAR,         // Constant annual reduction
        DECLINING_BALANCE // Percentage of remaining value per year
    }
}
```

```

}

struct CreditParams {
    // Immutable (set at creation)
    address insurer;           // Vendor who can update this credit
    uint8  assetClass;         // Asset classification (immutable)
    uint48 createdAt;          // Creation timestamp

    // Insurer-mutable (reappraisal, schedule changes)
    uint256 faceValue;         // Maximum insured value (18 decimals)
    uint256 depreciationFloor; // Minimum value after depreciation

    DepreciationType depType;   // Depreciation model
    uint32  depRate;            // Annual rate in basis points (10000 = 100%)
    uint48  depStartAt;         // When depreciation begins

    uint32  premiumRate;        // Annual premium: basis points of activated value
    uint48  lastUpdated;        // Timestamp of last insurer update

    // Client-mutable (activation)
    uint256 activatedValue;     // Currently activated portion (<= faceValue)
    uint48  lastActivatedAt;    // Timestamp of last activation
}

mapping(uint256 => CreditParams) public credits;
uint256 private _nextTokenId;

// --- Events ---
event CreditCreated(uint256 indexed tokenId, address indexed insurer,
                    address indexed owner, uint256 faceValue);
event CreditUpdated(uint256 indexed tokenId, address indexed insurer,
                    uint256 newFaceValue, uint32 newDepRate, uint32 newPremiumRate);
event CreditActivated(uint256 indexed tokenId, address indexed owner,
                     uint256 additionalValue, uint256 totalActivated);
}

```

Key design decisions:

- **insurer** is stored per-token (not a global role), allowing multiple insurers in the same contract.
- **assetClass** is immutable – the insurer can reappraise a house’s value but cannot reclassify it.
- **activatedValue** tracks the client’s cumulative execution, capped at **faceValue**.

3.2 Depreciation Models

Depreciation is computed deterministically from the NFT’s on-chain parameters. The insurer pays gas to update parameters; the **currentValue()** view function costs no gas to query.

```

/// @notice Current depreciated value of the activated portion of this credit.
/// @dev Pure computation from on-chain state -- no oracle needed.
function currentValue(uint256 tokenId) public view returns (uint256) {
    CreditParams storage c = credits[tokenId];
    if (c.activatedValue == 0) return 0;

    uint256 depreciatedFace = _depreciate(
        c.faceValue, c.depType, c.depRate,
        c.depreciationFloor, c.depStartAt
    );

    // Activated portion depreciates proportionally
    return depreciatedFace * c.activatedValue / c.faceValue;
}

```

```

function _depreciate(
    uint256 faceValue,
    DepreciationType depType,
    uint32 depRate,          // basis points per year
    uint256 floor,
    uint48 startAt
) internal view returns (uint256) {
    if (depType == DepreciationType.NONE || block.timestamp <= startAt) {
        return faceValue;
    }

    uint256 elapsed = block.timestamp - startAt;
    uint256 depreciable = faceValue - floor; // portion subject to depreciation

    if (depType == DepreciationType.LINEAR) {
        // loss = depreciable * rate * elapsed / (365.25 days * 10000)
        uint256 loss = depreciable * depRate * elapsed / (365.25 days * 10000);
        if (loss >= depreciable) return floor;
        return faceValue - loss;
    }

    if (depType == DepreciationType.DECLINING_BALANCE) {
        // value = floor + depreciable * (1 - rate/10000) ^ years
        // Use continuous approximation: exp(-rate * elapsed / (10000 * 365.25 days))
        // Implemented via fixed-point exp approximation (see _expNeg below)
        uint256 exponent = uint256(depRate) * elapsed / (365.25 days);
        // exponent is in basis-point-years; convert to 18-decimal fixed point
        // exponent_fp = exponent * 1e18 / 10000 = exponent * 1e14
        uint256 expFp = exponent * 1e14;
        uint256 factor = _expNeg(expFp); // returns 1e18 * exp(-expFp/1e18)
        return floor + depreciable * factor / 1e18;
    }

    return faceValue; // fallback
}

/// @dev Fixed-point exp(-x) for x in 18-decimal format.
/// Uses 6th-order Taylor series, accurate to <0.01% for x < 3.0 (ie. <300% total depreciation).
/// For production, use PRBMath.exp() or ABDKMath64x64.
function _expNeg(uint256 x) internal pure returns (uint256) {
    // exp(-x) ~ 1 - x + x^2/2 - x^3/6 + x^4/24 - x^5/120 + x^6/720
    uint256 UNIT = 1e18;
    if (x > 10 * UNIT) return 0; // effectively fully depreciated

    uint256 x2 = x * x / UNIT;
    uint256 x3 = x2 * x / UNIT;
    uint256 x4 = x3 * x / UNIT;
    uint256 x5 = x4 * x / UNIT;
    uint256 x6 = x5 * x / UNIT;

    // Positive terms: 1 + x^2/2 + x^4/24 + x^6/720
    uint256 pos = UNIT + x2 / 2 + x4 / 24 + x6 / 720;
    // Negative terms: x + x^3/6 + x^5/120
    uint256 neg = x + x3 / 6 + x5 / 120;

    if (neg >= pos) return 0;
    return pos - neg;
}

```

3.2.1 Depreciation Examples

Asset	Type	Rate	Floor	30yr value (of \$100K)
Land	NONE	–	–	\$100,000
House (bldg)	LINEAR	200bp	20%	\$52,000
Vehicle	DECLINING_BALANCE	1500bp	5%	\$5,633
Farm equip	DECLINING_BALANCE	1000bp	10%	\$16,791
Gold in vault	NONE	–	–	\$100,000

A residential property would typically have *two* BUCK_CREDITS: one for the land (NONE, full face value indefinitely) and one for the structure (LINEAR or DECLINING_BALANCE, with a floor representing salvage/lot-clearing value).

3.3 Activation (Execution) Model

The client executes (activates) their credit piecemeal. Each activation increases the `activatedValue` at the *current* premium rate set by the insurer. The premium is paid in BUCKs to the insurance pool.

```
/// @notice Client activates additional credit, up to the current face value.
/// @param tokenId The BUCK_CREDIT NFT to activate
/// @param amount Additional value to activate (18 decimals)
/// @dev Caller must be the token owner.
/// Premium for the newly activated amount is collected by BUCK.mint(),
/// not here -- activation just increases the limit.
function activate(uint256 tokenId, uint256 amount) external {
    require(ownerOf(tokenId) == msg.sender, "Not credit owner");
    CreditParams storage c = credits[tokenId];
    require(c.activatedValue + amount <= c.faceValue,
        "Exceeds face value");

    c.activatedValue += amount;
    c.lastActivatedAt = uint48(block.timestamp);

    emit CreditActivated(tokenId, msg.sender, amount, c.activatedValue);
}

/// @notice Aggregate current value of all credits owned by an account.
/// @dev Called by BUCK.mint() to compute the credit limit.
function totalCurrentValue(address account) external view returns (uint256) {
    uint256 total = 0;
    uint256 count = balanceOf(account);
    for (uint256 i = 0; i < count; i++) {
        total += currentValue(tokenOfOwnerByIndex(account, i));
    }
    return total;
}
```

The activation model separates the *decision to use credit* from the *act of minting BUCKs*. A client can activate \$200K of their \$500K home credit today, and months later activate \$100K more – possibly at a different premium rate if the insurer has updated the NFT in the interim.

3.4 Insurer Updates

The insurer can reappraise the asset, change depreciation parameters, and adjust premium schedules. These changes affect *future* credit limit computations, including already-activated amounts (since the depreciation function is computed from current parameters, not historical ones).

```

/// @notice Insurer updates credit parameters (reappraisal, schedule change).
/// @dev Only callable by the credit's designated insurer.
///      Cannot change assetClass or insurer address.
function updateCredit(
    uint256 tokenId,
    uint256 newFaceValue,
    uint256 newDepreciationFloor,
    DepreciationType newDepType,
    uint32 newDepRate,
    uint48 newDepStartAt,
    uint32 newPremiumRate
) external {
    CreditParams storage c = credits[tokenId];
    require(msg.sender == c.insurer, "Not insurer");

    // If face value decreases below activated, cap activated
    if (newFaceValue < c.activatedValue) {
        c.activatedValue = newFaceValue;
    }

    c.faceValue = newFaceValue;
    c.depreciationFloor = newDepreciationFloor;
    c.depType = newDepType;
    c.depRate = newDepRate;
    c.depStartAt = newDepStartAt;
    c.premiumRate = newPremiumRate;
    c.lastUpdated = uint48(block.timestamp);

    emit CreditUpdated(tokenId, msg.sender,
        newFaceValue, newDepRate, newPremiumRate);
}

```

Note that if the insurer reduces `faceValue` below the current `activatedValue`, the activated amount is capped downward. This could push the client's outstanding BUCK balance above their credit limit, triggering the default insurance mechanism described below.

4 BUCK: ERC-20 Token

The BUCK token is a standard ERC-20 with a custom `mint()` function that enforces credit limits. BUCKs are freely transferable once minted – they represent fungible value backed by the aggregate pool of attested assets, not any specific BUCK_CREDIT.

4.1 `mint()` Implementation

```

// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract Buck is ERC20 {

    BuckCredit public immutable buckCredit;
    BuckKController public immutable buckK;
    address public immutable insurancePool;

    uint256 constant PRECISION = 1e18;

    // Per-account stored credit limit (monotonically increases per mint)
    mapping(address => uint256) public storedLimit;

    event Minted(address indexed account, uint256 amount, uint256 premium,
        uint256 creditValue, uint256 buckKValue, uint256 newLimit);
}

```

```

constructor(address _buckCredit, address _buckK, address _insurancePool)
    ERC20("Alberta Buck", "BUCK")
{
    buckCredit = BuckCredit(_buckCredit);
    buckK = BuckKController(_buckK);
    insurancePool = _insurancePool;
}

/// @notice Mint BUCKs against the caller's aggregated BUCK_CREDIT value.
/// @param amount Desired BUCK amount to mint (18 decimals).
function mint(uint256 amount) external {
    // 1. Aggregate all BUCK_CREDITS in the caller's account
    uint256 totalCreditValue = buckCredit.totalCurrentValue(msg.sender);

    // 2. Get current BUCK_K stabilization factor
    uint256 currentBuckK = buckK.compute();

    // 3. Compute maximum credit limit
    uint256 maxLimit = totalCreditValue * currentBuckK / PRECISION;

    // 4. Only increase the stored limit (never decrease via mint)
    if (maxLimit > storedLimit[msg.sender]) {
        storedLimit[msg.sender] = maxLimit;
    }

    // 5. Check that mint doesn't exceed limit
    uint256 currentBalance = balanceOf(msg.sender);
    require(currentBalance + amount <= storedLimit[msg.sender],
        "Exceeds credit limit");

    // 6. Compute default insurance premium
    uint256 premium = _computePremium(
        msg.sender, amount, storedLimit[msg.sender]
    );

    // 7. Mint: net amount to client, premium to insurance pool
    _mint(msg.sender, amount - premium);
    if (premium > 0) {
        _mint(insurancePool, premium);
    }

    emit Minted(msg.sender, amount, premium,
        totalCreditValue, currentBuckK, storedLimit[msg.sender]);
}

/// @notice Burn BUCKs to reduce outstanding balance (repayment).
/// @dev Reduces balance, freeing credit capacity for future minting.
function burn(uint256 amount) external {
    _burn(msg.sender, amount);
}

```

4.2 Default Insurance Premium

The premium scales with utilization: the closer the account's outstanding BUCKs approach the credit limit, the more expensive each additional BUCK becomes. This mirrors the architecture document's description of Parametric Default Insurance.

```

/// @dev Premium as a fraction of the minted amount, scaling with utilization.
/// Uses a quadratic curve: premium_rate = baseRate + (utilization^2) * scaleRate
/// At 0% utilization: ~0.5% premium
/// At 50% utilization: ~1.75% premium
/// At 90% utilization: ~4.55% premium
/// At 100% utilization: impossible (can't mint past limit)
uint256 constant BASE_RATE = 50; // 0.50% base (in basis points)
uint256 constant SCALE_RATE = 450; // 4.50% additional at 100% utilization
uint256 constant BP = 10000;

```

```

function _computePremium(
    address account,
    uint256 mintAmount,
    uint256 limit
) internal view returns (uint256) {
    if (limit == 0) return 0;

    // Utilization after this mint
    uint256 newBalance = balanceOf(account) + mintAmount;
    uint256 utilization = newBalance * PRECISION / limit; // 0 to 1e18

    // premium_rate = BASE_RATE + utilization^2 * SCALE_RATE / 1e18
    uint256 utilSq = utilization * utilization / PRECISION;
    uint256 rate = BASE_RATE + utilSq * SCALE_RATE / PRECISION;

    return mintAmount * rate / BP;
}

```

The quadratic scaling creates a natural disincentive against over-leveraging. A client using 50% of their credit pays roughly 1.75% of each minted BUCK to the insurance pool; at 90% they pay 4.55%. The insurance pool accumulates these premiums to cover defaults (accounts where asset depreciation or BUCK_K changes push outstanding BUCKs above the credit limit).

5 BUCK_K: On-Chain PID Value Stabilization Controller

The BUCK_K contract maintains purchasing-power parity between the BUCK and its commodity basket. It implements a PID (Proportional-Integral-Derivative) controller identical in structure to the project's ownercredit PID controller, ported to Solidity fixed-point arithmetic.

5.1 Process Variable and Setpoint

PID Term	Source	Meaning
Process Variable	BUCK/USDC Uniswap V3 TWAP	What a BUCK actually trades for
Setpoint	Commodity basket oracle sum	What a BUCK <i>should</i> be worth (basket cost)
Error	Setpoint - Process	Positive = BUCK undervalued, negative = overvalued
Output (BUCK_K)	PID computation	Credit limit multiplier (1.0 = neutral)

When BUCK trades *below* basket value (positive error), BUCK_K increases, expanding everyone's credit limit. This floods the market with cheap BUCKs, which arbitrageurs buy to acquire underpriced commodities, driving the BUCK price back up. The converse reduces BUCK_K when BUCKs are overvalued.

5.2 Solidity Implementation

```

// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity ^0.8.20;

import "@chainlink/contracts/src/v0.8/interfaces/AggregatorV3Interface.sol";

contract BuckKController {

    // --- PID Gains (governance-set, 18-decimal fixed point) ---
    int256 public Kp;
    int256 public Ki;

```

```

int256 public Kd;

// --- PID State (updated on compute() when dT has elapsed) ---
int256 public P;          // Proportional error
int256 public I;          // Integral accumulator
int256 public D;          // Derivative
uint256 public lastUpdate; // Timestamp of last PID state update

// --- Output ---
uint256 public buckK;      // Current BUCK_K output (18-decimal, 1e18 = 1.0)
uint256 public dT;         // Minimum seconds between PID state updates

// --- Output Limits (anti-windup) ---
uint256 public buckKMin;   // Floor (e.g., 0.5e18 = 50% of neutral)
uint256 public buckKMax;   // Ceiling (e.g., 2.0e18 = 200% of neutral)

// --- Oracle Configuration ---
address public buckUsdcPool; // Uniswap V3 BUCK/USDC pool
uint32 public twapInterval;  // TWAP window in seconds (e.g., 1800 = 30min)

// Commodity basket: Chainlink price feeds + weights
struct BasketComponent {
    AggregatorV3Interface feed; // Chainlink price feed
    uint256 weight;             // Weight in basket (18 decimals, sum = 1e18)
    uint8 feedDecimals;         // Decimals of the Chainlink feed
}
BasketComponent[] public basket;

int256 constant UNIT = 1e18;
address public governance;

event BuckKUpdated(uint256 newBuckK, int256 error, int256 P, int256 I, int256 D);
event GainsUpdated(int256 Kp, int256 Ki, int256 Kd);

constructor(
    int256 _Kp, int256 _Ki, int256 _Kd,
    uint256 _dT,
    uint256 _buckKMin, uint256 _buckKMax,
    address _buckUsdcPool, uint32 _twapInterval,
    address _governance
) {
    Kp = _Kp; Ki = _Ki; Kd = _Kd;
    dT = _dT;
    buckKMin = _buckKMin;
    buckKMax = _buckKMax;
    buckUsdcPool = _buckUsdcPool;
    twapInterval = _twapInterval;
    governance = _governance;
    buckK = uint256(UNIT); // Start at 1.0 (neutral)
    lastUpdate = block.timestamp;
}

/// @notice Compute and return the current BUCK_K value.
/// @dev If dT has elapsed since last update, performs a full PID cycle
///      (reads oracles, updates P/I/D state, stores new buckK).
///      Otherwise returns the cached buckK value.
///      Called by BUCK.mint() -- the minter pays gas for any PID update.
function compute() external returns (uint256) {
    if (block.timestamp - lastUpdate < dT) {
        return buckK;
    }

    // --- Read oracles ---
    int256 basketCost = _getBasketCost(); // Setpoint: USDC cost of basket
    int256 buckPrice = _getBuckPrice();   // PV: BUCK market price in USDC

    // --- PID computation ---
    int256 dt = int256(block.timestamp - lastUpdate);

```

```

int256 error = basketCost - buckPrice; // Positive = BUCK undervalued

int256 newP = error;
int256 newI = I + error * dt / UNIT; // Scaled integral
int256 newD = dt > 0
    ? (error - P) * UNIT / dt // Rate of change
    : int256(0);

int256 rawOutput = UNIT // Start at 1.0 (neutral)
    + newP * Kp / UNIT
    + newI * Ki / UNIT
    + newD * Kd / UNIT;

// --- Anti-windup clamping ---
uint256 newBuckK;
if (rawOutput < int256(buckKMin)) {
    newBuckK = buckKMin;
    // Only update integral if it would push output away from saturation
    if (newI > I) I = newI;
} else if (rawOutput > int256(buckKMax)) {
    newBuckK = buckKMax;
    if (newI < I) I = newI;
} else {
    newBuckK = uint256(rawOutput);
    I = newI;
}

P = newP;
D = newD;
buckK = newBuckK;
lastUpdate = block.timestamp;

emit BuckKUpdated(newBuckK, error, P, I, D);
return newBuckK;
}

/// @notice Current BUCK_K without updating state (view-only).
function currentBuckK() external view returns (uint256) {
    return buckK;
}

// --- Oracle Helpers ---

/// @dev Sum of weighted commodity prices from Chainlink feeds, in USDC (18 decimals).
function _getBasketCost() internal view returns (int256) {
    int256 total = 0;
    for (uint i = 0; i < basket.length; i++) {
        BasketComponent storage comp = basket[i];
        (, int256 price,,) = comp.feed.latestRoundData();
        // Normalize to 18 decimals
        int256 normalized = price * int256(10 ** (18 - comp.feedDecimals));
        total += normalized * int256(comp.weight) / UNIT;
    }
    return total;
}

/// @dev BUCK/USDC price from Uniswap V3 TWAP oracle (18 decimals).
/// Uses the pool's observe() function for manipulation resistance.
function _getBuckPrice() internal view returns (int256) {
    // Uniswap V3 TWAP implementation:
    // 1. Call pool.observe([twapInterval, 0])
    // 2. Compute tick = (tickCumulatives[1] - tickCumulatives[0]) / twapInterval
    // 3. Convert tick to price: price = 1.0001^tick * 10^(token1Decimals - token0Decimals)
    //
    // Omitted here for brevity -- production would use Uniswap's OracleLibrary.
    // See: https://docs.uniswap.org/contracts/v3/guides/providing-liquidity/get-a-price
    revert("_getBuckPrice: implement with OracleLibrary");
}

```

```

// --- Governance ---

function setGains(int256 _Kp, int256 _Ki, int256 _Kd) external {
    require(msg.sender == governance, "Not governance");
    Kp = _Kp; Ki = _Ki; Kd = _Kd;
    emit GainsUpdated(_Kp, _Ki, _Kd);
}

function setDT(uint256 _dT) external {
    require(msg.sender == governance, "Not governance");
    dT = _dT;
}

function addBasketComponent(address feed, uint256 weight, uint8 decimals) external {
    require(msg.sender == governance, "Not governance");
    basket.push(BasketComponent({
        feed: AggregatorV3Interface(feed),
        weight: weight,
        feedDecimals: decimals
    }));
}

```

5.3 PID Tuning Considerations

The PID gains determine how aggressively BUCK_K responds to price deviations:

Gain	Effect of Increasing	Risk of Over-tuning
Kp	Faster response to price deviations	Oscillation, over-correction
Ki	Eliminates steady-state error (persistent mispricing)	Integral windup, slow recovery
Kd	Dampens oscillation, anticipates trends	Noise amplification

Initial gains should be conservative (high Kd relative to Kp, low Ki) to favor stability over responsiveness. The dT parameter (minimum update interval) acts as an additional dampening mechanism – a 1-hour dT means BUCK_K changes at most once per hour regardless of how many mints occur, preventing high-frequency oscillation.

The TWAP interval (`twapInterval`) provides manipulation resistance: a 30-minute TWAP means an attacker would need to sustain a price manipulation for 30 minutes to meaningfully affect BUCK_K – a prohibitively expensive proposition on a liquid pool.

5.4 Adaptive Gain Scheduling (Future Enhancement)

The current implementation uses governance-set fixed gains. A future enhancement could implement on-chain gain scheduling where Kp, Ki, and Kd are adjusted based on observed error dynamics:

- **Low volatility regime** (error variance below threshold): reduce Kp, increase Ki for tighter long-term tracking.
- **High volatility regime** (error variance above threshold): increase Kp and Kd for faster response, reduce Ki to prevent windup.
- **Regime detection**: maintain a rolling variance of the error signal over a window of N updates.

This maps directly to the `PIDController.reset()` pattern in the JavaScript implementation, where gains can be hot-swapped without spiking the derivative term.

6 Integration: Full mint() Walkthrough

Consider Alice, who owns a \$500K Calgary home (land: \$200K, structure: \$300K). Two insurers (LandCo, HomeCo) have issued BUCK_CREDITS:

Token	Insurer	Face Value	Depreciation	Activated	Current Value
#42	LandCo	200,000	NONE	200,000	200,000
#43	HomeCo	300,000	LINEAR, 200bp, 20% fl	150,000	141,000
Total					341,000

The structure has depreciated 6% over 3 years (LINEAR at 2%/yr applied to the 80% depreciable portion). Alice activated all of the land credit but only half of the structure credit.

Alice calls `BUCK.mint(50000e18)`:

1. `buckCredit.totalCurrentValue(alice)` returns `341,000e18`.
2. `buckK.compute()` fetches oracle prices:
 - Commodity basket costs 1.02 USDC (slight inflation above 1.00 parity).
 - BUCK trades at 0.99 USDC on Uniswap.
 - $\text{Error} = 1.02 - 0.99 = +0.03$ (BUCK undervalued by 3%).
 - PID output: $\text{BUCK_K} = 1.015$ (credit expansion to correct undervaluation).
3. $\text{maxLimit} = 341,000 * 1.015 = 346,115$ BUCKs.
4. Alice's current balance is 280,000 BUCKs. After mint: $330,000 < 346,115$. OK.
5. Utilization after mint: $330,000 / 346,115 = 95.3\%$.
 - Premium rate: $0.50\% + (0.953^2 * 4.50\%) = 4.59\%$.
 - Premium: $50,000 * 4.59\% = 2,295$ BUCKs to insurance pool.
6. Alice receives 47,705 BUCKs. Insurance pool receives 2,295 BUCKs.

Later, HomeCo reappraises the structure at \$280K (updated `faceValue`). Alice's credit limit drops. If her outstanding balance exceeds the new limit, the default insurance mechanism activates.

7 Gas Cost Analysis

Approximate gas costs (Ethereum mainnet, Solidity 0.8.x optimizer):

Operation	Gas (approx)	Cost at 30 gwei	Notes
<code>BUCK_CREDIT.activate()</code>	~50K	~\$4	Single SSTORE + event
<code>BUCK_CREDIT.updateCredit()</code>	~80K	~\$6	Multiple SSTOREs (insurer pays)
<code>BUCK.mint()</code> (no PID update)	~150K	~\$12	Enumerate credits + mint
<code>BUCK.mint()</code> (with PID update)	~350K	~\$28	+ Oracle reads + PID computation
<code>BUCK_CREDIT.currentValue()</code>	0 (view)	Free	Pure computation, no state change
<code>BUCK_K.currentBuckK()</code>	0 (view)	Free	Read cached value

The PID update cost ($\sim 200K$ gas additional) is amortized across all mints within a ΔT window. With a 1-hour ΔT , only the first mint after each hour pays for the oracle reads and PID computation.

For lower gas costs, the same contracts can be deployed on Ethereum L2s (Arbitrum, Optimism, Base) where gas is 10-100x cheaper, or on alternative EVM chains. The contract architecture is chain-agnostic.

8 Security Considerations

8.1 Oracle Manipulation

- **TWAP resistance:** The BUCK/USDC price uses a Uniswap V3 TWAP (30+ minutes), requiring sustained capital commitment to manipulate.
- **Chainlink feeds:** Commodity prices from Chainlink’s decentralized oracle network, with built-in deviation thresholds and heartbeat checks.
- **Staleness checks:** `_getBasketCost()` should verify `updatedAt` is within an acceptable window (e.g., 1 hour) and revert if stale.

8.2 Credit Enumeration DoS

A malicious user could create many tiny BUCK_CREDITS to make `totalCurrentValue()` gas-prohibitive. Mitigations:

- Minimum face value for BUCK_CREDIT creation.
- Cap on credits per account (e.g., 20).
- Off-chain aggregation with Merkle proof (advanced).

8.3 Insurer Trust

The insurer can reduce `faceValue` at any time, potentially pushing accounts into default. This is by design – it reflects real-world reappraisal risk. Mitigations:

- Insurance pool covers default events.
- Multiple competing insurers provide market discipline.
- Governance can blacklist abusive insurers.
- Time-locks on insurer updates (e.g., 7-day delay for reductions $> 20\%$).

8.4 Reentrancy

Standard OpenZeppelin `ReentrancyGuard` on `mint()` and `activate()`. All state changes occur before external calls.

9 Future Approaches

This ERC-721 implementation prioritizes clarity and correctness. Subsequent documents will explore:

9.1 ERC-1155: Semi-Fungible Batch Credits

If standardized credit classes emerge (e.g., "Calgary residential, HomeCo, LINEAR 200bp"), multiple properties in the same class could share a token ID with individual amounts. Benefits:

- Batch operations (`safeBatchTransferFrom`) for portfolio management.
- Reduced gas for accounts with many same-class credits.
- Natural fit for insurance tranches within a single credit.

Challenges: per-token depreciation start dates require additional state mapping, partially negating the fungibility benefit.

9.2 ERC-3643 (T-REX): Regulated Security Token

For production deployment under regulatory oversight:

- **ONCHAINID**: Native identity registry for KYC/AML compliance.
- **Compliance modules**: Programmable transfer restrictions (e.g., credits can only transfer to verified Alberta residents).
- **Agent roles**: Insurer as agent with granular permissions (update, freeze, force-transfer).
- **Recovery**: Lost-key recovery through identity verification.

This is the likely production standard if Alberta implements BUCK under provincial regulation.

9.3 ERC-4626: Vault-Based Insurance Pool

While a poor fit for BUCK_CREDIT itself, ERC-4626 is well-suited for the `InsurancePool`:

- Premium deposits as `deposit()` / mutual insurance shares as `shares`.
- `convertToAssets()` / `convertToShares()` for transparent share pricing.
- Standard DeFi composability (yield aggregators, lending protocols).

9.4 L2 and Cross-Chain Deployment

The contracts are EVM-compatible and can deploy on Arbitrum, Optimism, Base, or Polygon with minimal changes. Cross-chain BUCK transfers via canonical bridges or LayerZero/Axelar would enable multi-chain liquidity while maintaining a single BUCK_K oracle on the settlement layer.